

День 9 • 🎯 Лучшие практики AI-агентов, HITL, fallback

📋 Обсуждаемые темы:

- 🚀 Лучшие практики для AI-агентов и Deep Agents
- ⚠️ Что может пойти не так?
- 🔄 Fallback: резервные сценарии
- 👤 HITL (Human-in-the-loop)
- 🛡️ Обработка ошибок и устойчивость

Лучшие практики для AI-агентов: HITL и fallback

- AI-агенты — часть продакшн-систем
- Модели могут ошибаться, «галлюцинировать» или зависать
- Цель: надёжные, проверяемые и гибкие сценарии

 **Deep Agents:** дают готовую основу — встроенные планирование, HITL и сжатие контекста помогают быстрее выйти на надёжный сценарий без сборки графа с нуля.

⚠️ Что может пойти не так?

- 😞 Неверный или «придуманный» ответ модели
- 😓 Ошибка API, таймаут, rate limit
- 🤔 Внешние данные устарели или недоступны
- 🤖 Агент завис или зациклился

📌 Устранить все ошибки нельзя → перехватываем, подстраховываем, логируем.

💡 **Deep Agents:** встроенное сжатие контекста снижает риск переполнения; subagents изолируют сбои в подзадачах; меньше своей логики — проще добавлять `retry` и `fallback`.

Что такое fallback?

Fallback — резервный сценарий при сбое основного пути.

Варианты:

-  Упрощённая генерация или шаблонный ответ
-  Повтор запроса (retry) с backoff
-  Другая модель или провайдер
-  Передача человеку (HITL)

 **Deep Agents:** сценарий «пауза → решение человека» настраивается через `interruptOn` по имени инструмента — без отдельного графа и ручного resume.

Пример задачи с fallback

Агент: ответы по документации компании (Google Docs, GitHub).

Реализовать:

-  Внешний контекст (RAG)
-  Проверка уверенности → fallback на шаблон
-  Логирование источника и конфиденциальность

 **Deep Agents: Skills** — для подключения контекста/документации; `interruptOn` — для передачи сомнительного ответа человеку; виртуальная ФС — для выгрузки больших ответов и снижения переполнения контекста.

Fallback-промпты: пример

Стандартный: «На основе предоставленной информации дай точный ответ.»

Fallback: «Если данных недостаточно — скажи, что информации нет, и предложи обратиться к человеку.»

 Снижает риск галлюцинаций.

 **Deep Agents:** те же инструкции закладываются в `systemPrompt` ; при переполнении контекста срабатывает встроенная суммаризация — диалог сжимается без потери ключевых решений.



Что такое HITL (Human-in-the-loop)?

Человек участвует в решении, когда ИИ не справляется или не уверен.

Применение:

-  Критичные финансовые и медицинские системы
-  Обучение и менторство
-  Дообучение моделей по обратной связи

Ситуация	Действие HITL
Низкая уверенность модели	Передача ментору
Конфликтующие данные	Подтверждение пользователя
Ручной режим	Запрос в поддержку



Что такое Deep Agents

Deep Agent — это готовый «умный агент» поверх LangGraph.

- **LangGraph** — конструктор: вы сами рисуете граф (узлы, переходы), сами добавляете планирование, файлы, паузы для человека. Всё гибко, но всё нужно писать самому.
- **Deep Agents** — агент «из коробки»: одна функция `createDeepAgent`, модель + инструменты + промпт. Планирование, работа с файлами, подзадачи и HITL уже встроены и работают вместе.

Итого: в LangGraph вы собираете агента из кубиков; в Deep Agents кубики уже собраны — подключаете модель и свои инструменты.



LangGraph vs. Deep Agents

Возможность	LangGraph	Deep Agents
Планирование (разбить задачу на шаги)	Пишете узел и логику сами	Встроенный инструмент <code>write_todos</code>
Файлы (чтение, запись, поиск)	Реализуете сами	Готовые <code>read_file</code> , <code>write_file</code> , <code>ls</code> , <code>grep</code> и др.
Подзадачи (отдать работу «дочернему» агенту)	Строите вложенный граф сами	Инструмент <code>task</code> , настраиваемые subagents
Пауза для человека (HITL)	Прерывания и resume в графе сами	Параметр <code>interruptOn</code> и встроенная обработка
Длинный диалог (сжатие контекста)	Узел и логика сами	Встроенное middleware

- Графы агентов (LangGraph и др.) — удобная основа для fallback и HITL
- Deep Agents — пример «готового» агента с fallback- и HITL-возможностями поверх LangGraph
- Guardrails и валидация выхода снижают риски
- Наблюдаемость (логи, трейсы) — обязательна для отладки и доверия

- Надёжный агент = **границы ответственности** + fallback + HITL
- Fallback и HITL повышают доверие и стабильность
- Deep Agents позволяют быстрее внедрить этот контур: встроенные HITL, планирование и сжатие контекста уменьшают объём своей реализации

Ответственность за стабильность системы — на разработчике.

 Спасибо за внимание!

-  Исследуйте — пробуйте код, экспериментируйте
-  Углубляйтесь — каждая тема открывает новые возможности
-  Создавайте — даже маленький проект лучше одной теории

Успехов! 